

Optimistic Commit-and-Challenge Verification：分布式推理可验证协议设计

沉淀日期：2026-08-07

来源：对 VeriEdge 原机制（fixed-sketch TSTC）的重新设计讨论

状态：设计讨论稿（非最终协议；含待评估项）

1. 新系统设计（提议原文）

用户输入提示词，假设现在有 3 个节点。不再采取预先规定每个节点提交一份包含坐标点的张量（activation）的摘要，而是要求他们对整个张量做哈希（因为同设备同精度同样的模型，算出来的 activation 是可以逐比特相等的）。然后把这个哈希的结果提交。

如果用户觉得回答的质量明显下降或者异常，发起争议挑战。这里默认信任验证者。

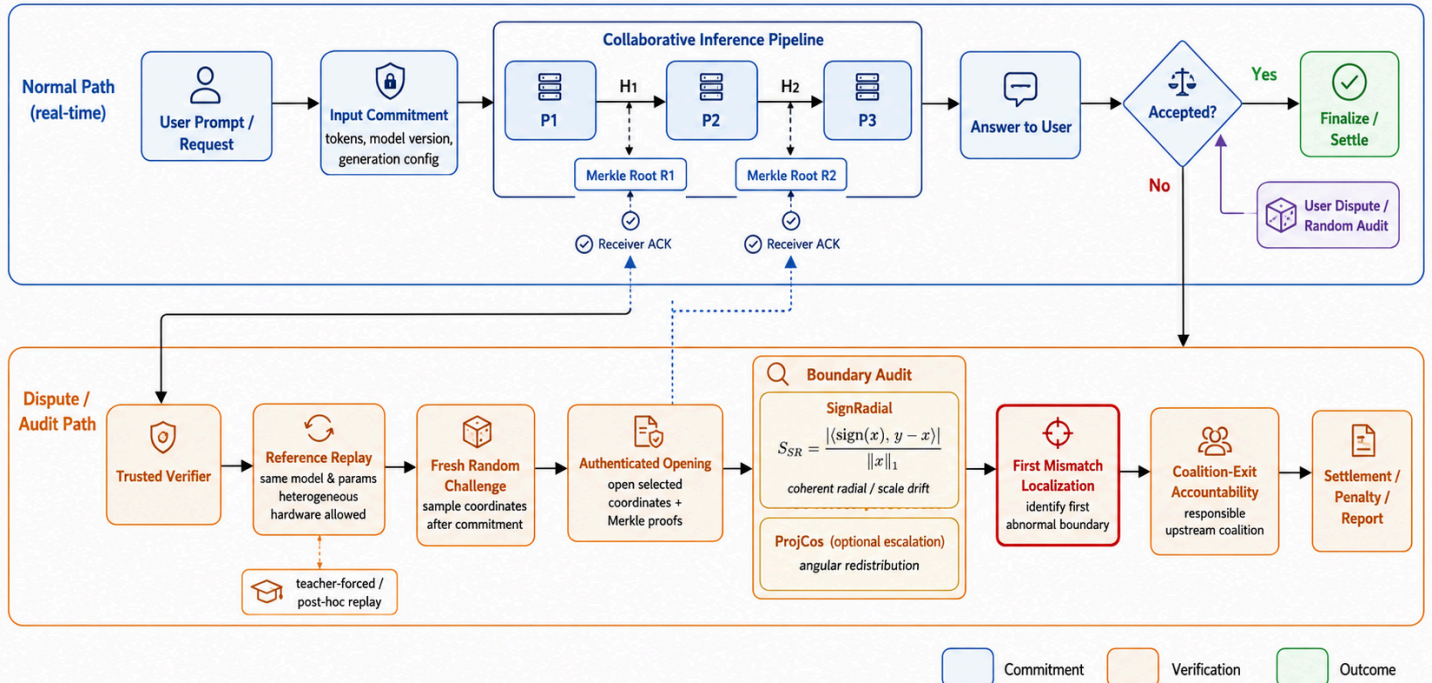
验证者会获取用户一开始的提示词或者 prompt 的 embedding，然后向第一个节点请求重放。因为第一个节点对整个 activation 做了哈希，所以假设他造假，他不能够在重放的时候装作诚实、放上正确推理的结果，不然匹配不上。

拿到了 activation 之后，验证者自己再运算一次得到自己的 activation。由于验证者很难使用跟推理节点完全一样的硬件配置（但是我们使用 SignRadial 和 projcos 算法，做到了误报率为 0，检测率 100%），所以验证者可以使用不同的设备、但是同模型同参数来运算 activation，然后秘密抽取一些点（坐标）来检测。虽然有数值漂移，但是依然能够检测是否异常。从而可以检测出来恶意节点篡改模型配置、直接修改 activation、使用精度更低的模型等等的恶意行为。

从而构成一套完整的端到端的可验证的分布式推理系统。

Optimistic Commit-and-Challenge Verifiable Collaborative Inference

System + SignRadial + Fresh Post-Commit Audit



核心流程

用户输入 prompt

↓

3 个节点 P1 → P2 → P3 分片执行

↓

每个节点对整个 activation 做哈希提交 (同设备同精度同模型 → 逐 bit 相等)

↓

用户拿到回答

↓

若回答质量明显下降 / 异常 → 用户发起争议挑战

争议流程

1. 验证者获取用户原始 prompt 的 embedding；
2. 向第一个节点请求重放 (或打开已提交的 activation)；
3. 验证者自己用**不同硬件、同模型同参数**重算 activation；
4. 秘密抽取坐标，用 SignRadial/ProjCos 检测；
5. 虽有异构数值漂移，但能检测是否异常 (篡改模型配置、直接改 activation、用更低精度模型等)。

信任假设

- 默认信任验证者（trusted verifier）。
- 检测恶意行为类型：篡改模型配置、直接修改 activation、使用精度更低的模型等。

2. 新旧方法对比

2.1 原机制（VeriEdge TSTC）

- 执行前固定 sketch policy / seed context / checked boundaries；
- 节点提交 **compact sketch digest**（不是完整张量）；
- 挑战时 reveal 固定 sketch；
- 执行前攻击者可能预知被检查的坐标。

2.2 新机制（Optimistic Commit-and-Challenge）

维度	原 TSTC	新机制
提交时机	执行前固定 sketch policy/seed	执行后 commit 完整边界张量哈希
提交内容	compact sketch digest	完整 activation 的哈希
挑战证据	固定 sketch	fresh 随机坐标 + 秘密抽样
攻击者预知坐标	可能（执行前固定）	不能（ commit 后才生成抽查坐标）
验证者设备	需与执行路径匹配	可用异构设备重算（SignRadial/ProjCos 容忍漂移）
正常路径开销	执行前校准 + sketch 生成	仅每边界一次流式哈希

2.3 新机制的核心优势

1. **Commit-before-inspect**：攻击者无法在执行前知道哪些坐标会被检查——比原 fixed-sketch 更强。
2. **异构验证者可行**：不要求逐 bit 相等，SignRadial/ProjCos 在异构硬件上容忍数值漂移（误报率 ≈ 0 、检测率 $\approx 100\%$ ）。

3. **首次挑战即可定位**：P1 作恶 $\rightarrow$ H1 首先异常；P1 诚实、P2 作恶 $\rightarrow$ H1 正常而 H2 首先异常 (first-mismatch localization)。
4. **正常路径几乎零验证开销**：只有对完整 activation 的一次流式哈希提交，不进入回答 latency critical path。

3. 计算开销分析

3.1 Routine path（正常路径）

最大新增开销不是 **SignRadial**，而是对完整 **activation** 的哈希提交 ($O(|H|)$, memory-bandwidth bound)。

关键优化：activation 本来就必须从 P_i 发送给 P_{i+1} 。不要"读 $H \rightarrow \text{Hash} \rightarrow$ 再发送"，而是在 **activation** 经过通信 **buffer** 时顺便 **streaming hash**。新增 memory traffic 大幅减少。

正常路径额外开销 $\approx$ 每个 **boundary** 一次流式哈希，不进入回答 latency critical path。

3.2 Challenge path（争议路径）

$$C_{\text{verify}} \approx C_{\text{replay}} + C_{\text{reveal}} + C_{\text{comparison}}$$

- **SignRadial** 比较 = $O(q)$ (如 $q=256$)，几乎可忽略。
- 真正昂贵的是验证者重跑一次推理，但只在争议/随机审计发生。
- 若 challenge probability p_c 很小 (如 1%)，平均成本 $E[C] = C_{\text{commit}} + p_c \times C_{\text{replay}}$ —— **expensive verifier replay** 只摊到 $\sim 1\%$ 任务。
- 用户先拿到答案；challenge/settlement 可异步完成，不进 critical path。

3.3 关键优化

1. **Verifier replay 用 teacher-forced forward**：验证者已知 $[\text{prompt}, y_1..y_T]$ ，可一次性 causal teacher-forced forward 获得全部 token 的 hidden states，**不需要逐 token autoregressive decode**。大幅降低重放成本。
2. **Activation retention 是 ephemeral**：节点只需保留到 challenge window (settlement/timeout 后删除)，不是永久存储。

4. Merkle Tree 搭建

新机制需要对完整 activation 做可验证的哈希提交。为了支持"挑战时秘密抽查少数坐标", 建议使用 **Merkle / chunk commitment**:

4.1 为什么用 Merkle 而不是普通哈希

若节点提交 $h = \text{SHA256}(H)$, 挑战后验证者秘密选 256 个坐标要求披露——验证者无法仅凭 **SHA256(H)** 证明这 256 个值属于之前哈希的完整 tensor。

两个方案:

- **方案 A: 披露整个 activation**——验证者检查 $\text{SHA256}(H)=h$ 后取坐标。安全成立, 但通信 $O(|H|)$ 。
- **方案 B (推荐): Merkle commitment**——把 activation 划成 chunks, 构建 Merkle tree, 正常执行只提交 root (~32 bytes); 挑战后验证者产生 fresh 随机坐标, 节点只打开对应 chunks + Merkle proof。

4.2 构建过程

```
activation H (shape 如 [T, D] 或 [B, L, H])
  ↓ 展平 + 分块
chunk_1, chunk_2, ..., chunk_N (每块固定字节数, 如 4KB 或 64 个 float)
  ↓ 逐块哈希
leaf_1 = H(chunk_1), ..., leaf_N = H(chunk_N)
  ↓ 逐层合并
internal = H(leaf_1 // leaf_2), ... (两两配对, 向上合并)
  ↓ 根
RH = MerkleRoot(H) (~32 bytes, 正常执行提交这个)
```

4.3 挑战时的 opening

验证者产生 fresh 随机坐标 Ω (如 $|\Omega| = 256$)

↓

节点打开对应坐标所属的 chunks: $\text{chunk}_i, i \in \text{covered}(\Omega)$

↓

附上 Merkle proof: 每个 chunk 的 sibling hashes (从 leaf 到 root 的路径)

↓

验证者 $\text{Verify}(\text{RH}, \text{chunk}_i, \text{proof}_i) = 1$

↓

从打开的 chunks 中取 256 个坐标值, 做 SignRadial 比较

Merkle 的关键性质:

- 正常执行只提交 32-byte root ($O(1)$ 通信)。
- 挑战时只打开被抽查的 chunks + proof ($O(\log N)$ per chunk), 远小于整个 tensor。
- **Commit-before-knowing-inspected-coordinates**: root 在坐标生成前已提交, 攻击者无法对未抽查坐标单独构造假值。
- 恶意节点若在抽查坐标上造假, Merkle proof 无法通过 root 校验。

4.4 搭建注意事项

- **chunk 大小**: 权衡打开粒度和 proof 大小。chunk 越小, 抽查粒度越细, 但 proof 越长 (树越深)。建议 chunk 覆盖一个坐标块 (如 8/16/64 个连续坐标), 与 SignRadial 的抽样对齐。
- **流式构建**: activation 在通信 buffer 中可边传边算 leaf hash, 避免二次读取。
- **坐标与 chunk 对齐**: SignRadial 抽样的坐标应映射到明确的 chunk 边界, 避免一个坐标跨两个 chunk。

5. SignRadial 方法: 详细计算

5.1 定义

SignRadial ($p=1$ radial coefficient) 在给定一组抽样坐标上, 度量**候选 activation** 相对**参考 activation** 的"相干幅值变化":

$$S_1 = \frac{|\sum_{i \in \Omega} \text{sign}(x_i) \cdot (y_i - x_i)|}{\sum_{i \in \Omega} |x_i|}$$

(纯文本等价: $S1 = | \sum \text{sign}(x_i)(y_i - x_i) | / \sum |x_i|$)

其中:

- x_i = 参考 activation 在第 i 个坐标上的值 (验证者重算得到)
- y_i = 候选/被挑战 activation 在第 i 个坐标上的值 (节点打开 commitment 得到)
- Ω = 验证者秘密抽样的坐标集合, 大小 $|\Omega| = q$ (如 256)
- $\text{sign}(x_i) = x_i$ 的符号 (+1 / -1 / 0)

5.2 直觉: 它测的是"误差是否与参考相干"

令残差 $e_i = y_i - x_i$ 。分子是残差在 $\text{sign}(x)$ 方向上的符号加权投影。两类误差在分子里行为完全不同:

- **scale 攻击 (相干):** $y = \alpha x$, 则 $e_i = (\alpha - 1)x_i$, 于是 $\text{sign}(x_i) \cdot e_i = (\alpha - 1) \cdot |x_i|$ 全部同号, 零抵消, 累加成 $(\alpha - 1) \cdot \sum |x_i|$, 除以分母精确得 $|\alpha - 1|$ 。
- **honest 量化噪声 (不相干):** 误差正负混合、与 x 的符号无系统相关, $\sum \text{sign}(x_i) \cdot e_i$ 是零均值随机游走, 正负相互抵消。

所以 SignRadial 不是测"误差有多大", 而是测"误差有多少沿 activation 自身方向"——scale/精度作弊是相干 (累积), honest 异构噪声是不相干 (抵消)。

5.3 逐坐标计算 (实现细节)

对 checkpoint 激活张量 $H \in \mathbb{R}^{\{T \times D\}}$ (如 $T=16$ tokens, $D=1024$ hidden):

Step 1 — 秘密抽样坐标

```
seed = PRF(K, request_id || layer_id)      # 每次请求独立, 验证者才知道
flat = RNG(seed).choice(T*D, size=q, replace=False)  # 从 T*D 个坐标中抽 q 个
rows, cols = unravel(flat, (T, D))         # 还原成 (token, channel) 索引
```

Step 2 — 读取坐标值

```
x = H_ref[rows, cols]      # 参考 activation (验证者重算)
y = H_cand[rows, cols]     # 候选 activation (节点打开 commitment)
```

Step 3 — 两个累加器 (P 和 B)

```

P = 0
B = 0
for i in range(q):
    diff = y_i - x_i          # 1 减法
    P += sign(x_i) * diff      # 1 符号判断 + 1 条件加减
    B += abs(x_i)              # 1 abs + 1 加法

```

Step 4 — 判定

```

score = |P| / (B + eps)      # 或部署时免除法: |P| > Γ·B
alarm = (score > Γ)          # Γ 为校准阈值

```

每坐标开销：1 减法 + 1 符号判断 + 1 条件加减 + 1 abs + 1 累加 $\approx$ **5 ops/坐标**，无乘法平方、无余弦、无归一化、无随机投影矩阵。 $q=256$ 时约 1,280 ops/张量，是 ProjCos4 (65,536 MACs) 的 **1/50**。

5.4 阈值校准 (Γ 如何确定)

Γ 只在 **calibration 数据**上校准 (与评估数据分离)：

```

对每个 checkpoint 边界 k:
    在 N 个 calibration 异构对上 (ref=左栈, cand=右栈, 同 prompt):
        计算每个对的 S1 原始值 (|P|/(B+eps))
    取这些值的 p99 分位数作为 Γ_k
可选: 乘以一个安全系数 (如 1.5-3.0) 以压低误报

```

- **逐层校准：**Γ 必须按 $\Gamma_{\{\text{model}, \text{layer}, \text{backend}\}}$ 分开——实测 honest p99 跨层差 **8.7×** ($C1=0.061$ vs $C3=0.007$)，单一全局阈值会误报或漏检。
- **判定规则：**候选对任一检查边界的 $S1 > \Gamma_k$ 即报警，首个超阈边界即 first-mismatch 定位。

5.5 关键性质

1. **scale 攻击精确响应：** $S1 = |\alpha-1|$ ($1.10\times \rightarrow 0.10$, $0.90\times \rightarrow 0.10$)，与抽哪些坐标无关，无随机投影 seed 风险。
2. **honest noise floor $\propto 1/\sqrt{q}$ ：**分子随机游走 $\text{std} \propto \sqrt{q}$ ，分母 $\propto q$ ，所以 honest S1 随 q 增加而下降 (实测 $p99(16)/p99(256)=4.13\approx 4$)；scale 信号恒定不随 q 变。 **$q=256$ 时 honest p99 (0.06) 远低于 scale 信号 (0.10)。**
3. **同时覆盖方向和数值：** $\text{sign_flip } (y=-x) \rightarrow S1=2$ ；方向变化攻击 TPR=1.0 (实测)。
4. **无乘法、免除法部署：** $|P| > \Gamma \cdot B$ 直接比较，适合在线融合进 activation 接收流程。

5. 盲点 (需 ProjCos 补): $\text{sign}(x)^T e = 0$ 使分子精确为 0; balanced per-channel scaling 使 $+c|x|$ 与 $-c|x|$ 抵消。这两类 ProjCos 能检测 (TPR 0.985-1.0), 因此双头互补。

6. SignRadial vs Scalar vs ProjCos — 实验对比

6.1 自适应攻击 (检测能力, 已测)

基础攻击族 (gaussian / scale / stale / layer_skip / low-precision) 的正式对比已统一到 §6.4 (720 池, 统一 1% FPR + Combined)。本节只保留 §6.4 未覆盖的自适应/白盒攻击结果——它们支撑 §5.5 的"sign-balanced 盲点需 ProjCos 补"声明。

自适应攻击 (held-out 200 prompts):

攻击	Scalar16	Scalar64	ProjCos4	SignRadial
sign-balanced ($\text{sign}(x)^T e = 0$)	0.015	0.135	0.985	0.0
balanced-channel ($\alpha=\{0.8,1.2\}$)	0.490	0.900	1.0	0.005

对新机制的意义:

- SignRadial 是理想的 primary fresh-coordinate verifier: $O(q)$ 纯加减、无随机投影 seed、scale/精度作弊全检测 (§6.4)。
- ProjCos 是唯一能补 sign-balanced 的 (0.985-1.0) ——Scalar16 也盲 (0.015)。应作为 escalation 保留 (sign-balanced / angular redistribution 场景)。
- Scalar 整体最弱 (sign-balanced 盲, FPR 高)。

6.2 真实精度作弊 (W8A16 重做, 2026-08-08)

重大修正: 旧 §6.2 表 (SignRadial=0.00、ProjCos=0.40、Scalar=0.50 的 W8A16 列) 经审查存在方法缺陷 (honest 基线=同设备重跑 ≈ 0 、阈值用 A/B 异构 0.066 与零噪声 honest 不匹配、score 用全张量), 已用统一协议重做。新结论: **W8A16 可被 100% 检测**。

新协议 (run_w8a16_redo.py , 720 池):

- honest 阈值 = calibration 的 A/B 异构 honest score $Q_{\{0.99\}}$ (SR p99=0.009、ProjCos p99=0.0006)

- $\text{ref}=\text{H_A}(\text{MPS})$ 、 $\text{cand}=\text{W8A16}(\text{H_B})(\text{CUDA bnb int8 权重}) \rightarrow \text{D}(\text{H_A}, \text{A}(\text{H_B}))$
- $\text{score} = \text{q}=64 \text{ 采样 SignRadial} + \text{ProjCos4}$, trace max over C1/C2/C3

case	SignRadial	ProjCos4	Combined
honest (FPR)	0.0096	0.0154	0.0096
W8A16 (TPR@1%FPR)	1.0000	1.0000	1.0000

读表： 阈值 = calibration A/B honest 的 $Q_{\{0.99\}}$ (nominal 1% FPR)。honest 行 = FPR (实测 0.96%~1.54%)，W8A16 行 = TPR@1%FPR (100%)。两行共用同一阈值，直接可比。

分离度： mean SR honest=0.0038 vs W8A16=0.279 (73 $\times$)；mean ProjCos 0.00014 vs 0.105 (750 $\times$)。

结论（修正后）：

1. **W8A16 (weight-only int8) 可被 activation 级检测器 100% 检出**——真实 int8 权重的量化误差逐层传播到边界 activation，即使 activation 保持 fp16。旧结论"W8A16 共同漏检区"是错误阈值的假象。
2. W8A8/FP8/INT4 的旧结论 (全 1.0) 不受影响。
3. 诚实保留：此结论依赖 W8A16 量化误差确实传到边界 activation (本模型 Qwen3-0.6B 实测；不同模型/层深可能不同)。

6.3 开销对比

指标	Scalar16	Scalar64	ProjCos4	SignRadial-64	SignRadial-256
算术 ops/张量	~32	~128	65,536 MACs	~320	~1,280
activation 读取	16	64	16 $\times$ 1024=16,384	64	256
参考值字节 (BF16)	64B	256B	256B	128B	512B
matmul / 归一化 / cosine	无	无	有	无	无

SignRadial-256 是 ProjCos4 的 1/50 计算量，无 matmul/余弦/归一化/随机投影 seed，纯加减，部署时 $|P| > \Gamma \cdot B$ 免除法。

6.4 统一 FPR 下的公平主表（720 池，TPR@1%FPR + Combined）【新增 2026-08-08】

审计修正：§6.1 初版表格各方法用各自的 threshold，FPR 不统一（Scalar16=0.150、Scalar64=0.415、ProjCos4=0.020、SR=0.000），TPR 直接不可比。已用**统一 nominal FPR=1%**重做：每检测器从独立 calibration 的 honest score 分布取 $\tau_D=Q_{\{0.99\}}$ ，在 untouched test 上测 TPR@1%FPR，并加 **Combined（ProjCos4 OR SignRadial, joint-calibrated max-fusion）**。

实验设置（720 池，双机真实异构）：

- 数据：**stack_a_720（mini1 MPS bf16）+ stack_b_720（RTX6000 CUDA fp32）**，720 prompts（200 calib / 520 eval），真实跨设备异构捕获。
- 阈值：calibration(200) honest 的 $Q_{\{0.99\}}$ ；Combined = $\max(u_P, u_R)$ （smooth ECDF）在 calibration 联合分布取 $Q_{\{0.99\}}$ ，**FPR 保持 ~1%（非 OR 并集）**。
- 攻击全部从 H_B 构造、比较 $D(H_A, A(H_B))$ 。

detector	evalFPR	gaussian	scale 1.10×	stale	layer_skip	INT8	INT4
Scalar16	0.000	1.000	0.000	0.948	0.771	0.969	0.969
ProjCos4	0.015	1.000	0.015 ≈FPR	1.000	1.000	1.000	1.000
SignRadial	0.010	0.977	1.000 ✓	1.000	1.000	1.000	1.000
Combined	0.010	1.000	1.000	1.000	1.000	1.000	1.000

结论（对新机制的直接支撑）：

1. **SignRadial 必要性**：scale 下 Scalar16=0.0、ProjCos4=0.015（≈FPR，数学盲）、**SR=1.0**。
2. **Combined 增加攻击覆盖率**：所有攻击 TPR=1.0，覆盖每个单检测器盲点（scale←SR 补、gaussian/stale/skip/lp←ProjCos 补）。
3. **统计稳健性**：200 calib 让 $Q_{\{0.99\}}$ 稳定（projcos4 evalFPR 从 40 池的 0.11 降到 0.015，combined 从 0.12 降到 0.010）。

脚本：run_p0_main_table_720.py → results/p0_main_table_720.csv 。

6.5 P2: Scale strength sweep — TPR@1%FPR vs $|\alpha-1|$ 【新增 2026-08-08】

实验设置：从真实候选 H_B 构造 $H_{\text{attack}} = \alpha \cdot H_B$, $\alpha \in \{0.90, 0.95, 0.98, 0.99, 1.01, 1.02, 1.05, 1.10\}$, 统一 TPR@1%FPR (720 池, SR $q=1024$)。

$ \alpha-1 $	Scalar16	ProjCos4	SignRadial	Combined
0.010	0.000	0.015	0.63-0.70	0.41-0.55
0.020	0.000	0.015	1.000	1.000
0.050	0.000	0.015	1.000	1.000
0.100	0.000	0.015	1.000	1.000

解读：

- **ProjCos4** 全程平台 $\approx 1.5\%$ (=FPR) ——数学盲点被直接证明： $\cos(H_A R, \alpha H_B R) = \cos(H_A R, H_B R)$, 投影余弦对 scale 不变。
- **SignRadial** 单调上升, 2% 处跳升到 1.0——coherent radial signal ($(\alpha-1)H_B$ 全坐标同号)。
- 图 results/p2_scale_sweep.png 是"给 mentor 的一页图"。



脚本： run_p2_scale_sweep.py → results/p2_scale_sweep.csv + .png 。

6.6 P1: Radial-Angular 机制实验 — SR/ProjCos 分别测 radial / angular components 【新增 2026-08-08】

这是回答"是不是只有 scale 一个攻击体现 SR 优势"的机制实验。用与具体攻击名无关的几何构造, 证明 SR 与 ProjCos 分别测量 activation error 的 radial 与 angular 分量。

实验设置：

$r = H_B / \|H_B\|_F$ (径向), $a \perp r$ (随机正交)
 $\Delta(\theta) = p \cdot \|H_B\|_F \cdot (\cos\theta \cdot r + \sin\theta \cdot a)$
 $H_{\text{attack}} = H_B + \Delta(\theta), \quad \theta \in \{0^\circ, 15^\circ, 30^\circ, 45^\circ, 60^\circ, 75^\circ, 90^\circ\}$

- 所有攻击**相同 relative-L2 magnitude ρ** ，唯一变化是 radial↔angular 混合。
- **SR $q=1024$** （关键： $q=64$ 时 angular 端随机游走噪声掩盖盲点； $q=1024$ 才让正交扰动可靠 cancel）。
- $\rho=0.005$ （主档）——小 ρ 让 angular 残差低于 honest 阈值。

结果 ($\rho=0.005$, SR $q=1024$, TPR@1%FPR):

θ	ProjCos4	SignRadial	Combined
0° (纯 radial)	0.015 $\approx$ FPR (盲)	0.935 (强)	0.752
30°	0.946	0.450	0.902
90° (纯 angular)	1.000 (强)	0.567 (弱)	1.000

机制结论（方法层面的质变）:

- **ProjCos** 检测 **angular drift** ($\theta=0$ 盲 $\rightarrow \theta=90^\circ$ 升到 1.0)
- **SignRadial** 检测 **coherent radial drift** ($\theta=0$ 强 $\rightarrow \theta=90^\circ$ 降到 0.57)
- **Combined** 全区间覆盖——二者互补，不是偶然发现 SR 检测 $1.1\times\text{scale}$ ，而是 **SR 与 ProjCos 分别测量 activation perturbation 的两个几何分量**。
- **诚实边界**: SR 在 $\theta=90^\circ$ 非 0 (~ 0.57)，因有限采样下正交扰动不完全 cancel；SR 的**完全盲点**只对 $\text{sign}(x)^T\Delta=0$ 的攻击 (sign-balanced) 成立。

图（机制说明）:



脚本: `run_p1_radial_angular_v3.py` $\rightarrow$ `results/p1_radial_angular_v3.csv` + `.png`。

7. 安全边界（建议明确）

Goal	描述	SignRadial/ProjCos 能否保证
Goal A: 检测有害执行偏离	cheating $\rightarrow$ activation/output materially different	✅ 适合（实验验证）
Goal B: 证明节点按声明精度执行	即使 W8A16 数值接近也要处罚	⚠️ 待评估（activation 检查可能无法完整保证）

建议 **claim** 定为"**detect material execution divergence**", 而非"prove exact backend compliance" (后者需 TEE/attestation/weight commitment/VC)。

Challenge 触发

建议不要只依赖"用户觉得答案不对" (subtle attack 可能永不 challenge)。建议：

- **user dispute + cheap output judge + small random audit probability** (如很低概率随机审计)
- 作用: subtle corruption 也被抽查; 恶意节点不知道哪次一定不被验证。

8. 待评估项 (明确标记)

-  统一 FPR 下的公平主表 (§6.4: 720 池, TPR@1%FPR + Combined, 200 calib 稳定统计)
-  scale strength sweep (§6.5: TPR vs $|\alpha-1|$, ProjCos 平台、SR 单调上升)
-  radial-angular 机制实验 (§6.6: SR/ProjCos 分别测 radial/angular components)
-  **W8A16 重做** (§6.2: 统一协议下 weight-only int8 可 100% 检出, 推翻旧"漏检区"结论)
-  W8A8/FP8/INT4 需在统一协议下复核 (旧表用旧协议, 但结论方向不受影响)
-  坐标保密 (fresh-coordinate) 下攻击能力的定量评估 (初步试验结果暂不收录)
-  Merkle commitment 的原型实现与通信/计算实测
-  teacher-forced forward 重放优化的实测加速比
-  随机审计概率 pc 与安全性的权衡

9. 产物

workspace/SRR/

└─ docs/OPTIMISTIC_COMMIT_CHALLENGE_DESIGN.md # 本文件 (v2)